

HOUSTON CLOUD

De app de escritorio a servicio multi-tenant en GCP

Aislamiento por equipo a nivel de kernel, sin VPS dedicada, con costo controlado

Plan técnico + estimación de costos · Junio 2026

CONTEXTO

El objetivo en una frase

Hoy Houston es una app de escritorio (Tauri) donde cada usuario corre su propio engine local. Queremos ofrecerlo **hosteado en GCP**, donde cada equipo solo accede a sus agentes y la separación es de cómputo, no de código.

PUNTO 1

Subir a la nube (GCP)

- Dejar el escritorio.
- El engine corre en GCP.
- Web + PWA como clientes.
- Acceso desde cualquier lado.

PUNTO 2

Cada equipo, sus agentes

- Aislamiento por equipo.
- Aunque sea la misma org.
- Seguridad por membership.
- Nada de fugas entre teams.

PUNTO 3

Aislamiento barato

- Casi a nivel de hardware.
- Sandbox de kernel (gVisor).
- Sin VPS dedicada por team.
- Nodos compartidos = barato.

PUNTO 1

De escritorio a cloud en GCP

El engine ya es un binario standalone que habla HTTP/WS. No se reescribe; se hostea y se le construye el entorno cloud alrededor.

- **Engine standalone listo** — houston-engine, parametrizado por HOUSTON_HOME + token.
- **Cliente web reemplaza a Tauri** — mismas vistas de ui/; examples/smartbooks ya lo prueba.
- **PWA móvil ya existe** — mismo protocolo, solo cambia el baseUrl.
- **Login de proveedor headless** — device-code flow ya soportado (sin navegador local).
- **Base de imagen ya hecha** — always-on/ trae Dockerfile + compose + systemd.

Trabajo real: shell web + control plane + gateway de tenancy + provisioning con sandbox. El núcleo del engine no se toca.

PUNTO 2

Cada equipo solo ve sus agentes

El tenant es el EQUIPO. La frontera correcta NO es lógica.

Modelo	Frontera	Veredicto
Engine compartido + filtro por tenant_id	Código (lógica)	Un bug filtra datos. Es 'separar por prompt'.
Un proceso engine por equipo, mismo host	Proceso	Mejor, pero comparten kernel y disco.
Un engine por equipo en su sandbox kernel	microVM / gVisor	Elegido: escape = romper el kernel.

Decisión: tenant = team. Un houston-engine por equipo, en su propio sandbox, con su volumen y su token.

Org → Team → Agents. La org agrupa facturación/admin; el equipo es la unidad de aislamiento. El gateway valida membership antes de rutear; el token del engine nunca llega al cliente.

PUNTO 3

Aislamiento de kernel, barato, sin VPS dedicada

GCP no expone Firecracker directo, pero da aislamiento de kernel sobre nodos COMPARTIDOS:

DEFAULT PRUEBA

Cloud Run gen2

microVM + gVisor gestionada.
Scale-to-zero: team idle = \$0.
Ideal para uso interactivo.

DEFAULT PAGO 24/7

GKE + GKE Sandbox

Pod gVisor por equipo, namespace aislado.
Spot VMs + bin-packing = barato.
Soporta routines 24/7.

Por qué cumple lo que pediste:

- **Frontera de kernel** — gVisor/microVM, no un check de código. No es 'separar por prompt'.
- **Sin VPS por cliente** — muchos sandboxes bin-packeados en pocos nodos compartidos.
- **Económico** — Spot VMs + scale-to-zero (prueba) + un solo control plane compartido.

Componentes en GCP

CLIENTE

Web + PWA

@houston-ai/engine-client
Supabase JWT (Google SSO)

BORDE

Gateway / Router

Valida JWT + membership
Rutea user→team→engine

PLANO CONTROL

Provisioner

Orgs / teams / roles
Crea engines + volúmenes

METADATA

Postgres

orgs, teams,
memberships (RLS)

Data plane — un engine aislado por equipo:

TEAM A

Engine (gVisor)

HOUSTON_HOME vol A
DB + workspaces + .houston/
Token propio

TEAM B

Engine (gVisor)

HOUSTON_HOME vol B
Aislado de A
Token propio

TRANSVERSAL

Plataforma

Secret Manager (tokens/keys)
Artifact Registry (imagen firmada)
Logging / Monitoring / Audit
VPC + Cloud NAT (egress allowlist)

MODELO

Dos tiers: prueba (barato) y pago (fuerte)

	Prueba / Free	Pago / Pro
Sustrato	Cloud Run gen2	GKE + GKE Sandbox
Ejecución	On-demand, scale-to-zero	24/7 routines, min 1 réplica
Aislamiento	microVM + gVisor (¡se conserva!)	pod gVisor + namespace
Billing API	BYO key (la trae el team)	Central con markup + cuotas
Infra fija que usa	Solo Cloud Run + Supabase	GKE + NAT + Cloud SQL
Costo p/ nosotros	~\$0 idle · fijo ~\$22/mes	~\$5-8 /equipo · fijo ~\$186

Clave del ahorro — el tier de prueba quita GKE/NAT/Cloud SQL pero MANTIENE el sandbox de kernel. Promoción free→pro = mover el volumen, sin migrar datos.

Defensa en capas

1 · Identidad

Supabase + Google SSO (ya existe). JWT con claims org/team.

2 · Autorización

Gateway valida membership. El token del engine nunca sale del borde.

3 · Cómputo

Sandbox de kernel (gVisor) por team. Garantía dura, no un if.

4 · Red

NetworkPolicy deny-all este-oeste + egress allowlist por team.

5 · Secrets

Secret Manager por team, rotación, nunca en imagen ni logs.

6 · Datos reposo

Buckets/discos por team con CMEK. Path-traversal ya cubierto.

7 · Suministro

Artifact Registry + scan + Binary Authorization (solo firmadas).

8 · Auditoría

Cloud Audit Logs + tracing por request. Quién accedió a qué.

COSTOS

Estimación — supuestos

Orden de magnitud, región us-central1, USD/mes. Infra propia; el costo de API LLM va aparte (ver nota).

Fijo / compartido — Pago vs Prueba

Concepto	Pago	Prueba
GKE cluster fee	\$73	—
Cloud SQL (control plane)	\$35	—
Cloud NAT	\$33	—
Gateway + control plane (CR)	\$20	\$15
Supabase (metadata)	incl.	\$0*
Logging / Monitoring	\$20	\$5
Secret Manager + misc	\$5	\$2
TOTAL FIJO	≈ \$186	≈ \$22

* Uso ligero de prueba cae en la cuota always-free de Cloud Run = \$0 cómputo. Filestore (caro) solo opt-in en Pago.

Variable por equipo (infra, sin API)

Concepto	Pago 24/7	Prueba
Cómputo (Spot/CR)	\$3.0	\$0-2
Storage GCS	\$0.30	\$0.10
NAT / egress datos	\$1.0	~\$0
Logs / secrets / misc	\$0.70	~\$0
POR EQUIPO	≈ \$5-8	≈ \$0-2

API LLM: Prueba = BYO (costo \$0 para nosotros). Pago = pass-through + markup → ingreso positivo, no costo neto.

COSTOS

Escenarios y comparación

Escenario	Fijo	Variable	Total infra/mes	Por equipo
Solo prueba: 2,000 free (BYO)	\$22	~\$0 idle	≈ \$22-100	≈ \$0.02
50 equipos Pago	\$186	$50 \times \$7 = \350	≈ \$536	≈ \$10.7
200 equipos Pago	\$186	$200 \times \$6 = \$1,200$	≈ \$1,386	≈ \$6.9
100 Pago + 400 prueba	\$186	$\$700 + \sim \0	≈ \$886	≈ \$1.8
500 equipos Pago	\$186	$500 \times \$5.5 = \$2,750$	≈ \$2,936	≈ \$5.9

Vs. el modelo que rechazamos (1 VPS dedicada por equipo):

- **200 VPS dedicadas (e2-small on-demand)** ≈ \$2,600/mes solo cómputo — sin bin-packing ni scale-to-zero.
- **Nuestro modelo a 200 Pago** ≈ \$1,386/mes con aislamiento de kernel MÁS fuerte. ~2x más barato.
- **La economía mejora con escala** — mejor bin-packing baja el costo por equipo al crecer.

Roadmap por fases

0	Web shell	Frontend web sobre engine-client (sin Tauri). 1 engine remoto.
1	Tenancy + control plane	Orgs/teams/memberships + gateway JWT + provisioner.
2	Aislamiento híbrido	Cloud Run gVisor (prueba) + GKE Sandbox (pago) + volúmenes.
3	Hardening red + datos	Egress allowlist, NetworkPolicy, CMEK, Binary Authorization.
4	Billing dual + cuotas	BYO (prueba) + central markup (pago), medición y rate-limit.
5	Producto	Panel admin, roles, observabilidad por tenant, SLA.

RESUMEN

Decisiones tomadas y próximos pasos

Lo que ya está decidido:

- **Tenant = equipo** — un engine aislado por team, sandbox de kernel (no RBAC lógico).
- **Sustrato híbrido** — Cloud Run (prueba) + GKE Sandbox (pago 24/7) desde el inicio.
- **Billing dual** — BYO key en prueba, central con markup + cuotas en pago.
- **Caso central = 24/7 routines** — define Pago como producto; scale-to-zero en prueba.

Próximos pasos

1. Confirmar storage default del tier Pago (propuesta: GCS, Filestore opt-in).
2. Arrancar Fase 0: shell web sobre @houston-ai/engine-client.
3. Diseñar el esquema del control plane (orgs/teams/memberships + RLS).